杭州电子科技大学

《计算机网络实验》实验报告

课程号：(2023-2024-1)-S0507170-09

实验三　DNS 抓包及分析

指导老师：

姓名学号：

2026 年 11 月

一、实验目的

- 掌握 nslookup 工具的基本用法，能够用其查询 A、AAAA、NS、MX、PTR 等各类资源记录；
- 通过抓包分析 DNS 查询报文与响应报文的格式，理解事务 ID、标志位、问题区、资源记录区的具体字段含义；
- 验证 DNS 在传输层使用 UDP 且目的端口固定为 53，并确认本机使用的默认 DNS 服务器地址；
- 观察 DNS 缓存与本地域名服务器的递归解析过程，理解为什么多数查询的响应时间只有几毫秒；
- 理解 CNAME 别名记录在 CDN 场景下的作用，以及 PTR 反向解析在 nslookup 启动时的附带行为。

二、实验设备和环境

表 1 实验软硬件环境

项目	配置说明
客户端主机	Windows 11，IPv4 192.168.1.3（WLAN 网卡）
默认 DNS 服务器	192.168.1.1（家用无线路由器，同时是默认网关）
备用/外部 DNS	8.8.8.8（Google Public DNS，用于替代已停服的测试服务器）
查询工具	nslookup（Windows 自带），命令行执行
抓包工具	Wireshark 3.6.16 / dumpcap
捕获过滤器	udp port 53 or tcp port 53
捕获文件	cap_dns.pcapng（48 个报文）、cap_dns2.pcapng（25 个报文，含 NS/MX 与 8.8.8.8 查询）

本实验的网络结构很简单：本机把全部 DNS 查询都交给默认网关 192.168.1.1，由路由器内置的 DNS 转发器（dnsmasq 一类程序）代为向上游递归或转发。因此在抓包时看到的所有查询的目的地址都是 192.168.1.1，而不是我们要查询的目标域名的权威服务器地址。这一点是理解整个实验结果的关键前提。

三、实验过程及步骤

3.1 抓包方法

DNS 报文通常很小，且查询往往在发起访问的瞬间完成，因此先用捕获过滤器限定端口 53，避免抓到无关流量：

```
> "E:\tools\Wireshark\dumpcap.exe" -i 6 -f "udp port 53 or tcp port 53" \
      -a duration:60 -w cap_dns.pcapng
```

3.2 查询亚洲站点

先用 nslookup 查询一个亚洲地区的站点，本实验选取韩国高丽大学主页：

```
> nslookup www.korea.ac.kr
服务器:  UnKnown
Address:  192.168.1.1

非权威应答:
名称:    www.korea.ac.kr
Address:  163.152.6.10
```

抓包结果中对应的报文为第 11、12 号（A 记录查询与响应）以及第 13、14 号（AAAA 记录查询与响应）。A 记录的应答中只有 1 条答案：163.152.6.10；AAAA 记录的应答中答案数为 0，即该站点没有配置 IPv6 地址。响应时间（dns.time）为 4.792 ms。

同时也可以查询国内站点作为对照：

```
> nslookup www.baidu.com
非权威应答:
名称:    www.a.shifen.com
Addresses:  36.152.44.93
            36.152.44.132
Aliases:  www.baidu.com
```

这里出现了一个重要的现象：应答中的名称被替换成了 www.a.shifen.com，而原查询名 www.baidu.com 反而出现在 Aliases 一行。这说明 www.baidu.com 是 www.a.shifen.com 的别名（CNAME），应答中共有 3 条资源记录：1 条 CNAME 记录加 2 条 A 记录。这正是 answers 计数为 3 的原因，也说明"一个查询名对应几个 A 记录"与"回答数为几"并不等价。

3.3 查询欧洲高校的权威名称服务器

要查询某域的权威名称服务器，需要指定记录类型为 NS：

```
> nslookup -type=NS mit.edu
非权威应答:
mit.edu nameserver = asia1.akam.net
mit.edu nameserver = use2.akam.net
mit.edu nameserver = use5.akam.net
mit.edu nameserver = eur5.akam.net
mit.edu nameserver = ns1-37.akam.net
mit.edu nameserver = ns1-173.akam.net
mit.edu nameserver = usw2.akam.net
mit.edu nameserver = asia2.akam.net
```

抓包中对应的报文为第 16、17 号，应答中 answers 计数为 8，与命令行输出的 8 条 nameserver 完全对应。这 8 台服务器全部属于 akam.net 域，说明 MIT 把权威 DNS 托管给了 Akamai 的托管 DNS 服务，而不是自行维护。从命名可以看出 Akamai 在全球部署

了多个节点：asia1/asia2 对应亚洲，eur5 对应欧洲，usw2 对应美国西部，use2/use5 对应美国东部，ns1-37 与 ns1-173 是编号节点。这种"同一域名的权威服务器分散在全球多地"的部署方式，正是 DNS 层面实现就近解析与容灾的基础。

3.4 查询邮件服务器

邮件服务器的记录类型为 MX，查询结果中同时给出优先级与邮件交换主机名：

```
> nslookup -type=MX yahoo.com
非权威应答:
yahoo.com       MX preference = 1, mail exchanger = mta5.am0.yahoodns.net
yahoo.com       MX preference = 1, mail exchanger = mta6.am0.yahoodns.net
yahoo.com       MX preference = 1, mail exchanger = mta7.am0.yahoodns.net
```

抓包中第 20、21 号报文对应此次查询，answers 计数为 3。三条记录的 preference 值都是 1，说明这三台邮件服务器处于同等优先级，发信方可以任选其一，从而起到负载分担的作用。与之对照的是，如果 preference 值不同，则数值小的优先，只有高优先级主机不可达时才会尝试下一台。

另外注意到，MX 记录给出的是主机名而不是 IP 地址，因此真正投递邮件前还需要再对该主机名做一次 A 记录查询。这是 DNS 设计中"记录可以互相引用"的体现。

3.5 用指定的 DNS 服务器查询

实验指导书原本要求用 bitsy.mit.edu 作为指定的名称服务器，但该主机早已停止对外提供 DNS 服务，实测执行 nslookup www.aiit.or.kr bitsy.mit.edu 会直接超时失败：

```
> nslookup www.aiit.or.kr bitsy.mit.edu
DNS request timed out.
    timeout was 2 seconds.
服务器:  UnKnown
Address:  192.168.1.3

*** 请求 UnKnown 超时。
```

为了完成该题的实验目的（即"向一个非默认的 DNS 服务器发起查询"），本实验改用 Google 的公共 DNS 8.8.8.8 作为替代：

```
> nslookup www.aiit.or.kr 8.8.8.8
DNS request timed out.
    timeout was 2 seconds.
服务器:  UnKnown
Address:  8.8.8.8

非权威应答:
名称:    www.aiit.or.kr
Addresses:  104.21.74.8
            172.67.152.120
```

抓包清楚地解释了这段输出中的"超时"从何而来。第 9 号报文是本机向 8.8.8.8 发起的 8.8.8.8.in-addr.arpa 反向查询，第 10 号报文在 37 ms 后成功返回 PTR 记录

dns.google，说明链路本身是通的。第 11 号报文是 A 记录查询，但它在抓包文件中没有任何对应的响应报文；第 12 号报文是随后的 AAAA 记录查询，第 13 号报文在 216 ms 后返回了 IPv6 地址。也就是说，超时只发生在 A 记录查询这一次，而 nslookup 在等待 2 秒后重试并最终拿到了结果，因此命令行的输出是"先报超时，再给出地址"。这一现象在不稳定的公共 DNS 上是偶发但常见的，恰好说明了 DNS 客户端需要具备重传与超时容忍机制。

3.6 观察到的一个附带现象：反向解析本机名

在整理抓包结果时发现，每一次执行 nslookup 之前，本机都会先发出一条针对 1.1.168.192.in-addr.arpa 的 PTR 查询，而路由器返回的是 rcode=3（No such name）加一条 SOA 记录。这个查询与我们要查的域名毫无关系，其真正原因是 nslookup 在启动时需要打印"服务器: xxx"这一行，它会尝试把 DNS 服务器地址反向解析成主机名；由于 192.168.1.1 在局域网内没有配置 PTR 记录，解析失败，于是命令行里显示的服务器名就是 UnKnown。理解这一点后，就不会把这 10 多条 PTR 查询误认为是实验内容的一部分。

3.7 DNS 报文的字段结构

在 Wireshark 中展开任意一条 DNS 报文，可以看到其结构由固定首部与若干区段组成（图 1）。

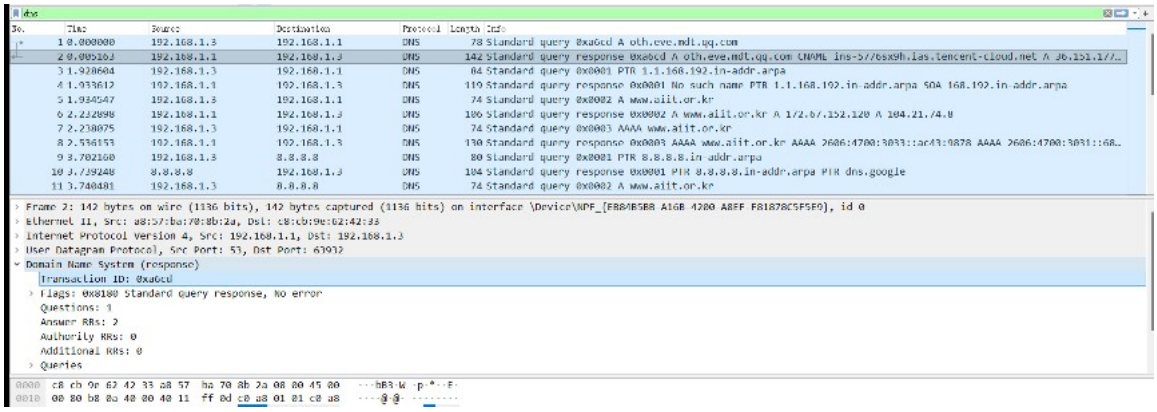


图 1 DNS 响应报文的字段结构（Transaction ID、Flags、各计数与资源记录）

表 2 DNS 报文首部与问题区、资源记录区的字段

字段	长度	含义
Transaction ID	16 位	事务 ID，由客户端随机生成，响应必须原样带回，用于把响应与请求配对
Flags	16 位	包含 QR（0 查询/1 响应）、Opcode（0 标准查询）、AA（权威应答）、TC（截断）、RD（期望递归）、RA（支持递归）、Rcode（0 无错 / 3 名字不存在）

字段	长度	含义
Questions	16 位	问题区记录数，正常查询为 1
Answer RRs	16 位	回答区资源记录数
Authority RRs	16 位	权威区记录数，否定应答时通常带 1 条 SOA
Additional RRs	16 位	附加区记录数
Queries	变长	查询名（按标签长度编码）＋查询类型（A/AAAA/NS/MX/PTR）＋查询类（IN 表示 Internet）
Answers	变长	资源记录：名称＋类型＋类＋TTL（32 位）＋数据长度＋数据

用本次抓到的响应报文举例：第 17 号报文（mit.edu 的 NS 查询响应）的 Flags 为 0x8180，展开后 QR=1（响应）、Opcode=0（标准查询）、AA=0（非权威）、TC=0（未截断）、RD=1（客户端期望递归）、RA=1（服务器支持递归）、Rcode=0（无错误）；Questions=1，Answer RRs=8，正好对应 8 条 NS 记录。再如第 4 号报文（本机名反向解析的响应）Flags 的 Rcode 为 3，Answer RRs=0，Authority RRs=1，是典型的否定应答。

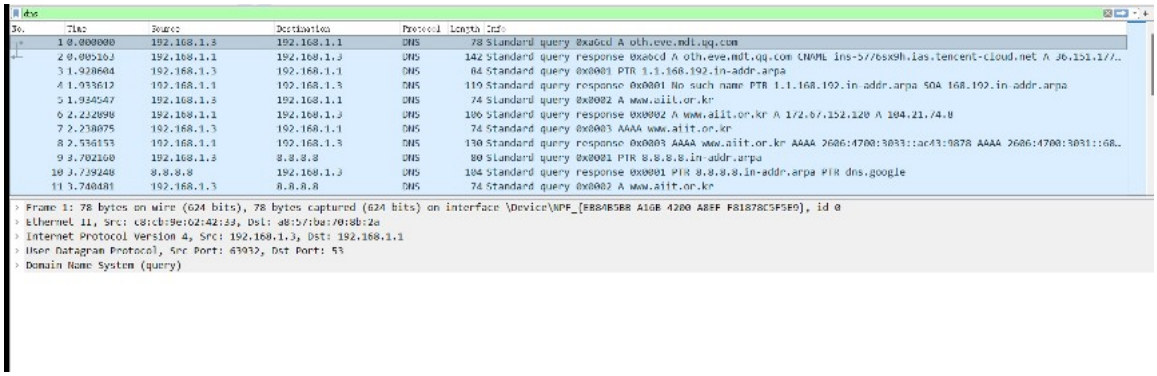


图 2 cap_dns2.pcapng 中的 DNS 报文列表（含 PTR、A、AAAA、NS、MX 各类查询）

四、实验问题解答

表 3 DNS 实验问题逐条解答

序号	问题	解答与依据
1	nslookup 查询亚洲站点（www.korea.ac.kr）得到的 IP 地址是什么？	163.152.6.10（第 11、12 号报文）。该地址属于韩国高丽大学，AAAA 查询的答案数为 0，说明该站点未提供 IPv6 地址。
2	查询欧洲某高校（mit.edu）的权威名称服务器，结果是什么？	nslookup -type=NS mit.edu 返回 8 台服务器：asia1.akam.net、asia2.akam.net、eur5.akam.net、use2.akam.net、use5.akam.net、usw2.akam.net、ns1-37.akam.net、ns1-173.akam.net。全部托管在 Akamai，说明该校未自建权威 DNS。
3	查询 yahoo.com 的邮件服务器，结果是什么？	mta5.am0.yahoodns.net、mta6.am0.yahoodns.net、mta7.am0.yahoodns.net，三者的 preference 均为 1，属于同优先级、可负载分担的三台服务器。
4	DNS 查询报文使用 UDP 还是 TCP？	使用 UDP。本次抓到的全部 DNS 报文（含 A、AAAA、NS、MX、PTR）的 IP 协议号均为 17，Wireshark 中 Protocol 列显示为 DNS，Transport 层为 User Datagram Protocol。只有

序号	问题	解答与依据
		当响应超过 512 字节或被截断（TC=1）时才会改用 TCP。
5	DNS 查询报文的目的端口是多少？	53。源端口为客户端随机分配的高位端口，本次抓包中先后使用 55617–55620、54554、56986、54320、52536–52538 等；响应报文的源端口为 53，目的端口回填为同一客户端端口。
6	DNS 查询报文发往哪个 IP 地址？该地址是否为本机的默认 DNS？	发往 192.168.1.1，与本机配置的默认 DNS 服务器地址完全一致。该地址同时是默认网关，说明家用路由器承担了 DNS 转发职责。唯一例外是 3.5 节特意指定的 8.8.8.8 查询。
7	DNS 查询报文中的查询类型（Type）是什么？	nslookup 对每个域名会先后发出两条查询：Type=A（1）取 IPv4 地址，Type=AAAA（28）取 IPv6 地址。另外 -type=NS 对应 Type=2，-type=MX 对应 Type=15，反向解析对应 Type=PTR（12）。
8	响应报文中提供了几个"answers"？	随域名而异：www.korea.ac.kr 为 1，www.baidu.com 为 3（1 条 CNAME + 2 条 A），www.mit.edu 为 3（2 条 CNAME + 1 条 A），mail.yahoo.com 为 3，mit.edu 的 NS 查询为 8，yahoo.com 的 MX 查询为 3，www.aiit.or.kr 的 A 与 AAAA 各为 2。
9	响应报文中的目的 IP 地址是否与发起查询时的目标 IP 一致？	一致。以 www.korea.ac.kr 为例，查询发往 192.168.1.1:53，响应由 192.168.1.1:53 发回 192.168.1.3 的同一临时端口。在 8.8.8.8 的实验中同样满足：查询发往 8.8.8.8:53，响应来自 8.8.8.8:53。若二者不一致，说明报文被中间设备伪造或重定向。
10	本机对同一域名是否发起了多次查询？为什么？	会。一是 A 与 AAAA 是两条独立查询；二是执行 nslookup 时若第一次超时会重试，例如向 8.8.8.8 查询时第 11 号 A 查询无响应，第 12 号再次发起 AAAA 查询。另外在多次实验过程中重复执行同一命令也会产生重复查询。
11	如果页面中嵌入了其他域名的对象，是否会重新进行 DNS 查询？	会。DNS 缓存以域名为粒度，不同域名即使指向同一台服务器也必须分别解析。本实验在实验二中访问 HTTP-wireshark-file4.html 时就观察到：主页面 gaia.cs.umass.edu 与图片服务器 kurose.cslash.net 是两个不同的域名，因此浏览器分别发起了 DNS 查询，得到的地址分别是 128.119.245.12 与 2.56.99.24。
12	为什么多数 DNS 响应的时延只有几毫秒？	因为命中了缓存。本地域名服务器（192.168.1.1）在 TTL 有效期内直接返回缓存结果，不需要逐级向根、顶级域、权威服务器递归。本次实测 192.168.1.1 的响应时间在 4.6–6.1 ms 之间，而向 8.8.8.8 查询时达到 216–298 ms，正说明了"缓存命中"与"跨网递归"的差距。
13	为什么响应中的名称与查询名不同（例如 www.baidu.com → www.a.shifen.com）？	因为该域名配置了 CNAME 别名记录。应答中先返回 CNAME 记录把查询名映射到规范名，再返回规范名的 A 记录。这是 CDN 常用的做法：在同一别名下按地域或负载动态返回不同的 A 记录。
14	nslookup 输出的"非权威应答"是什么意思？	表示该应答来自缓存或由非权威服务器转发，并非由该域的权威名称服务器直接给出，因此理论上可能不是最新的。本次所有应答的 AA 标志位均为 0，与命令行提示一致。
15	如何在抓包中找到某个特定域名的查询？	使用显示过滤器 dns.qry.name == "www.korea.ac.kr"，或直接 dns.qry.name contains "korea"。若要同时看请求与响应，可配合 dns.flags.response == 0 / 1 区分方向。
16	反向查询（PTR）在本次实验中出现	出现在每条 nslookup 命令之前，查询名为 1.1.168.192.in-

序号	问题	解答与依据
	在什么位置？作用是什么？	addr.arpa，即把 192.168.1.1 的地址反转后加上 in-addr.arpa 后缀。这是 nslookup 为显示"服务器:"名称而做的解析，由于局域网内未配置该 PTR 记录，响应为 rcode=3，因此在命令行中显示为 UnKnown。
17	DNS 报文的最大长度受什么限制？什么时候会改用 TCP？	UDP 方式下传统限制为 512 字节；启用 EDNS0 后可协商到更大值，但在部分网络环境下仍可能被截断。当响应超过限制时，服务器把 TC 标志置 1 并只返回部分内容，客户端随即改用 TCP 53 端口重新查询。本次实验查询的都是小记录类型，未触发 TC。
18	如何通过抓包判断 DNS 服务器是否支持递归？	看响应报文中的 RA（Recursion Available）标志位。本次所有来自 192.168.1.1 与 8.8.8.8 的响应 RA 均为 1，说明二者都支持递归查询；而请求报文中的 RD（Recursion Desired）也均为 1，表示客户端希望服务器代为递归。
19	把 TTL 从抓包中读出来，它与什么有关？	每条资源记录的 TTL 为 32 位整数，单位为秒，表示该记录在缓存中的有效期。例如 www.aiit.or.kr 的 A 记录 TTL 为 300 秒，这是 Cloudflare 为代理域名设置的较短缓存时间，便于在切换后端时快速生效；而 NS 记录的 TTL 通常为 3600 秒或更长。
20	为什么 NS 查询返回的是 8 条记录而不是 1 条？	因为一个域通常配置多台权威服务器以保证容灾；应答的 Answer 区可以容纳多条同类型记录。本次 mit.edu 的 8 条 NS 记录全部属于 akam.net，Akamai 自身再通过 anycast 把这些主机映射到全球多个节点。
21	指定名称服务器查询时（nslookup <域名> <服务器>）抓包有什么不同？	目的 IP 从默认 DNS 变为指定的服务器地址，并且 nslookup 会先向该地址发一条针对其自身地址的 PTR 查询，用于在命令行中显示服务器名称。本次向 8.8.8.8 查询时就抓到了 8.8.8.8.in-addr.arpa 的 PTR 查询与 dns.google 的应答。
22	实验指导书要求的 bitsy.mit.edu 无法使用，如何处理？	该主机已停止对外服务，nslookup 会返回 2 秒超时。替代方案是改用任意可用的公共 DNS（如 8.8.8.8、1.1.1.1），实验目的（向非默认服务器发起查询并观察报文）同样可以达到。报告中应如实说明替代原因，而不是伪造结果。
23	DNS 是否提供可靠性保证？如果响应丢失会怎样？	UDP 本身不保证可靠。当响应丢失时，DNS 客户端依靠"超时重传"来弥补：Windows 解析器的默认超时为 1 秒、重试 2 次，nslookup 显示为 "DNS request timed out. timeout was 2 seconds."。本次向 8.8.8.8 查询时第 11 号 A 报文确实没有响应，随后的重试才拿到结果，是这一机制的实证。

五、实验总结

5.1 遇到的问题与解决办法

表 4 DNS 实验中的常见问题与处理

问题现象	原因分析	解决办法
nslookup 输出开头总有一行"服务器: UnKnown"	nslookup 尝试把 DNS 服务器地址反向解析成主机名，局域网内无 PTR 记录	属正常现象，可在报告中说明；也可在执行前用 set type=a 减少干扰

问题现象	原因分析	解决办法
nslookup www.aiit.or.kr bitsy.mit.edu 直接超时	bitsy.mit.edu 已停止提供 DNS 服务	改用 8.8.8.8 等公共 DNS 完成"指定服务器查询"的实验目的
抓包文件里 DNS 报文远多于预期	本机后台程序（联想更新、微信、wpad 探测等）也在做 DNS 查询	用显示过滤器 dns.qry.name contains "目标域名" 精确筛选，或在捕获过滤器上再叠加 host 限定
向 8.8.8.8 查询时出现"先超时后成功"	A 记录查询的响应确实丢失，客户端重试后成功	在报告中把命令行输出与抓包对照说明，这正好是 DNS 超时重传机制的直接证据
www.baidu.com 的 answers 计数与看到的 IP 数量不一致	应答中包含 CNAME 记录，一条 CNAME 也计入 answers	用 dns.count.answers 与 dns.a 分别统计，或用 dns.cname 单独查看别名链

5.2 收获与体会

本实验让我第一次真切地看到"域名解析"这件事在网络上长什么样。在此之前，nslookup 只是一个敲完就出结果的命令；抓包之后才发现，一次看似简单的访问背后其实是一组结构严谨的报文：固定 12 字节的首部、按标签编码的查询名、以 TTL 标注有效期的资源记录。尤其是把 Flags 字段的 0x8180 拆成 QR/Opcode/AA/TC/RD/RA/Rcode 七组标志，再与命令行输出的"非权威应答""无错误"一一对应，这种从比特到语义的贯通是单纯背课本得不到的。

第二个收获是对"缓存"的量化认识。同一个查询，交给本地路由器只要 5 毫秒左右，交给 8.8.8.8 却要 200 毫秒以上，差了将近两个数量级。正是这个差距解释了为什么 DNS 缓存是互联网性能的关键一环，也解释了为什么 TTL 的设置需要在"更新及时"与"减少查询"之间权衡：TTL 太长则切换后端后旧地址仍在生效，太短则递归查询量激增。

第三个收获来自那个"失败"的 bitsy.mit.edu 实验。最初以为是抓包配置有问题，排查后才确认是远端服务器早已下线。这件事提醒我：网络实验依赖真实的外部环境，而这些环境是会变化的。遇到与指导书不符的结果时，正确的做法是先确认现象、再分析原因、最后用等效手段完成实验目的，并在报告中如实记录，而不是把结果改成"应该的样子"。